The Journal of Open Source Software

modepy: Basis Functions, Interpolation, and Quadrature (not just) for Finite Elements

Andreas Klöckner [1], **Alexandru Fikl** [2], **Jacob Xiaoyu Wei** [3], **Thomas H. Gibson** [4], **and Addison J. Alvey-Blanco**[1]

1 Siebel School of Computing and Data Science, University of Illinois at Urbana-Champaign, US **2** Institute for Advanced Environmental Research (ICAM), West University of Timișoara, Romania **3** Pathlit, US **4** Advanced Micro Devices Inc., US

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 03 September 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

modepy provides a means for constructing geometric shapes for reference elements, equipping them with appropriate approximation spaces, and numerically performing calculus operations (derivatives, integrals) on those spaces.

modepy focuses on high-order accuracy – given an element size h, this refers to the asymptotic decay of the approximation error as $O(h^n)$, for $n \geq 3$, assuming sufficient smoothness of the solution being approximated. If we consider a problem in d dimensions, the number of unknowns scales as $O(h^{-d})$. In the absence of high-order approximation, say at $n = 1$, even modest accuracy increases (e.g. an additional significant digit) can incur significant cost (a $1000\times$ increase in the problem size in 3D). Therefore, if accuracy is desired at manageable cost, high-order methods are crucial.

A popular approach for accurate approximation of functions on geometrically complex domains is the use of *unstructured discretizations*, which represent the geometry as a typically disjoint union (a "mesh") of primitive geometric shapes, most often simplices and quadrilaterals. Given the means to perform calculus operations on these *reference elements* and mapping functions from them to the *global* elements, calculus operations become available on the entire domain. Notably, one can approximate the reference-to-global coordinate mapping function itself with the same high-order machinery. Altogether, this paves the way for the solution of integral and (partial) differential equations. Those, in turn, can be used to model many phenomena in the physical world, including fluid flow, electromagnetism, and solid mechanics. The finite element method (FEM) is a popular example of this, including its continuous and discontinuous flavors. Further examples include collocation, spectral, and Nyström methods.

modepy has been used to construct FEM solvers (Glusa, 2021; Klöckner & others, 2025) and integral equation solvers (Kloeckner & others, 2025). It is written in pure modern Python 3, offering comprehensive type annotations and full documentation with minimal runtime dependencies, of which numpy is perhaps the most significant. Versions going back to as early as 2013 offer broad compatibility with older versions of Python, including Python 2.7.

modepy is licensed under the MIT license and available on Github at https://github.com/inducer/modepy/.

Statement of need

High-order accurate calculus operations on unstructured discretizations are crucial to many solvers of differential and integral equations, but their uses can also extend to computer graphics

and Computer-Aided Design (CAD). This functionality is often embedded in an ad-hoc manner in larger codes, leading to limited scope and a lack of reusability. `modepy` addresses this need by providing a robust, reusable, generalizable, and composable implementation. FIAT (Homolya et al., 2025) is an early software tool aiming to address a similar need. It offers a comprehensive set of finite elements and basis functions, but largely focuses on tabulation. FInAT (Ham et al., 2025), a somewhat recent refinement, adds a focus on algebraic expressions for basis functions, exposing details needed for efficient tensor product evaluation. Both are, however, tightly interwoven into the FENiCS/Firedrake ecosystems (e.g. by depending on UFL, the "Unified Form Language" used for the expression of variational forms). `StartUpDG.jl` (Chan et al., 2024) provides functionality with some overlap with `modepy`, but with a focus on the needs of discontinuous Galerkin FEM. `QuadPy` (Schlömer et al., 2021) also has some overlap with `modepy`, in that it can provide quadrature rules, and, while it has comprehensive coverage, it is regrettably no longer open-source, and it lacks `modepy`'s composability. `minterpy` (Wicaksono et al., 2025), meanwhile, deals exclusively with polynomial interpolation, with a focus on sparse grids.

A significant complicating factor in providing a flexible implementation is that numerical solvers for differential equations (and likely many other candidate users) must satisfy rigorous cost constraints, as simulation fidelity is always traded off against computational cost. As a result, such software commonly adopts cutting-edge high-performance computing (HPC) techniques, such as GPU computation and distributed-memory approaches. To facilitate separation of those concerns from the numerical methods, `modepy` adopts a two-pronged approach. In many cases, it suffices for operations to be represented as data in matrix or tabular form, so that no actual execution of `modepy` code is needed in a cost-constrained setting.

A prominent exception to this is the evaluation of basis functions. To facilitate this, `modepy` allows its function evaluation to be "traced" (in the sense of lazy/deferred evaluation), so that an expression graph can be obtained. This graph is represented with the help of the `pymbolic` (Kloeckner et al., 2024) software library. This, in turn, can interoperate with Python ASTs (Python Software Foundation, 2024), SymPy (Meurer et al., 2017), SymEngine (Čertík et al., 2013), as well as a number of other pieces of software for symbolic computation. Another case where a purely data-driven approach falls short is the setting of tensor-product elements. In this case, for example, a mass matrix $\boldsymbol{M}$ with entries

$$M_{ij} = \int_{[-1,1]^d} \phi_i(\boldsymbol{r})\phi_j(\boldsymbol{r})\mathrm{d}\boldsymbol{r}$$

permits a Kronecker product factorization

$$\boldsymbol{M} = \boldsymbol{M}^{1D} \otimes \cdots \otimes \boldsymbol{M}^{1D}, \qquad \text{with} \qquad M^{1D}_{ij} = \int_{[-1,1]} \phi_i(r)\phi_j(r)\mathrm{d}r,$$

assuming a suitable index order of nodes $\boldsymbol{r}_i$ and basis functions $\{\phi_i\}$. In d dimensions, the use of this factorization reduces the asymptotic complexity of a matrix-vector product $\boldsymbol{Mu}$ from $O(N_p^{2d})$ to $O(N_p^{d+1})$, where N_p is the number of degrees of freedom in one dimension, and d is the number of dimensions. To take advantage of this factorization, an abstraction such as `modepy`'s must expose additional information, including numbering of degrees of freedom. In `modepy`'s case, this is accomplished via reshaping operation that converts arrays of degrees of freedom back and forth between a flat (one-dimensional) and a structured (d-dimensional) representation. This reshaping is applicable to any array type (including GPU or other high-performance arrays) as long as `numpy`-compatible reshaping is supported.

Overview

The high-level concepts available in `modepy` are shapes (i.e the geometry of a domain), modes (i.e. the basis functions), and nodes (i.e. the degrees of freedom). These are im-

plemented in a user-extensible fashion using the `singledispatch` mechanism, with inspiration taken from common idiomatic usage in Julia (Bezanson et al., 2017). Given a function space `space` on a `shape`, a set of nodal degrees of freedom can be obtained via `edge_clustered_nodes_for_space(space, shape)`, with an implementation selected based on the type of `space`. While the example shows that multiple dispatch could be beneficial, we have decided to forgo it to avoid potential soundness issues (Nardelli et al., 2018).

Shapes

The geometry of a reference element is described in `modepy` by the `Shape` class. Built-in support exists for `Simplex` and `Hypercube` geometries, encompassing the commonly used interval, triangle, tetrahedron, quadrilateral, and hexahedral shapes (see Figure 1). `TensorProductShape` can be used to compose additional shapes, such as prisms, that are useful in specific applications and sometimes generated by meshing software (such as `gmsh` (Geuzaine & Remacle, 2009)). An example that constructs a prism and performs some standard operations on it is given in the next section.

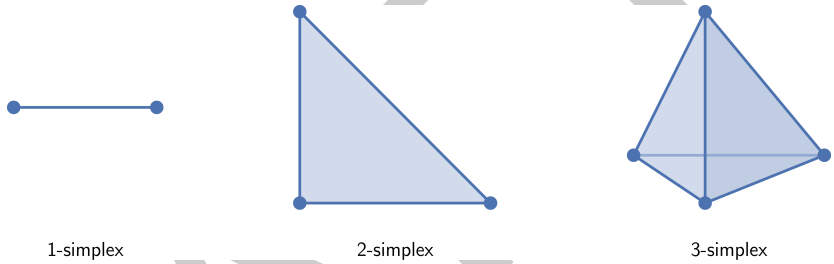

Figure 1: Domains corresponding to the one-, two-, and three-dimensional simplices.

To support a wide array of applications, shapes can be queried and manipulated in several ways. The `faces_for_shape` function can be used to return a list of faces, which are shapes themselves, that allow access to domain normals and mappings back to the reference element.

Modes and Spaces

To perform calculus operations, each reference element can be equipped with function spaces described by the `FunctionSpace` class. These represent a finite-dimensional space of functions $\phi_i : D \to \mathbb{R}$, where D is the reference element domain, but make no specific choice of basis. The function spaces mirror the shape definitions and are intended to work in tandem. In fact, the `space_for_shape` function provides a default choice of space for each shape. Predefined choices include the `PN` space, containing polynomials of total degree at most `order`, and the `QN` space, containing polynomials of maximum degree at most `order`. As with shapes, these spaces can also be combined using `TensorProductSpace` to describe functional spaces for other domains or in higher dimensions.

For calculations, each space can be equipped with a default basis using the `basis_for_space` function. `modepy` provides two special types of basis functions: orthonormal (accessed through `orthonormal_basis_for_space`) and monomial (accessed through `monomial_basis_for_space`). The basis is provided in the form of a `Basis` class, which allows access to the basis functions, their derivatives (per axis) and opaque `mode_ids`. As noted before, the basis functions and their derivatives can be evaluated directly or can be traced using the `pymbolic` library to aid in code generation.

DRAFT

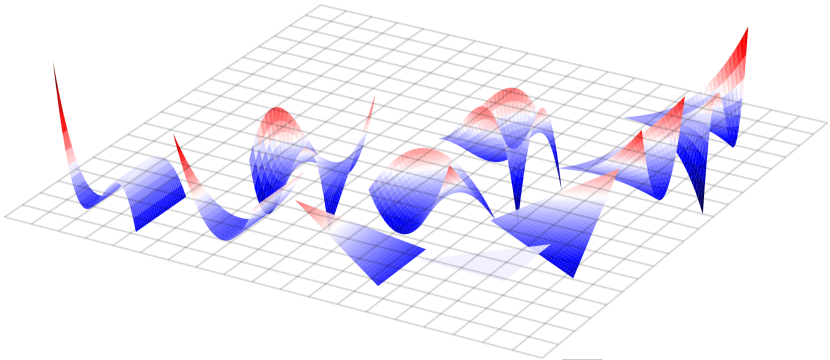

Figure 2: PKDO basis functions for the triangle.

By default, the `Simplex` shape is equipped with a `PN` functional space. This function space uses the Proriol-Koornwinder-Dubiner-Owens (PKDO) basis (see Figure 2) as defined in (Dubiner, 1991) for dimensions up to 3. The `Hypercube` is equipped with the `QN` functional space that uses a tensor product of standard Legendre polynomials. Additionally, general Jacobi polynomials are available and can be used as a basis for variants of `QN` with different weights, depending on the application.

Nodes

A final component in an FEM discretization (or 'Ciarlet triple' (Brenner & Scott, 2007, sec. 3.1)) is a set of 'degrees of freedom' ('DOFs'), that is, a vector of numbers (technically, functionals) that uniquely define a certain element in the span of a basis. The literature considers various types of degrees of freedom; the two most common are modal DOFs (i.e. basis coefficients) and nodal DOFs (i.e. function or derivative values at certain points). `modepy` provides easy access to both, while paying particular attention to their usability in the high-order setting. Due to Runge's phenomenon (Trefethen, 2020), simple equispaced nodal DOFs rapidly become unusable with increasing polynomial order due to poor conditioning. As such, `modepy` offers access to many types of nodes with better properties for each reference element. There also exists an ability to construct one's own set of nodes given a set of functions in 2D via a procedure based on eigenvalues of multiplication operators (Vioreanu & Rokhlin, 2014).

On each shape, three default families of nodes are provided: equispaced (accessed through `equispaced_nodes_for_space`), edge-clustered (accessed through `edge_clustered_nodes_for_space`) and randomly distributed (accessed through `random_nodes_for_space`).

For more generality, `modepy` can directly interoperate with the `recursivenodes` library described in (Isaac, 2020), which offers well-conditioned nodes on the simplex that obey a 'recursive' property, where nodes on a face of a higher-dimensional shape are exactly the lower-dimensional nodes. In addition, it offers many many tunable parameters (e.g. nodes on the boundary vs. not). On simplices, the "warp-and-blend" nodes (Warburton, 2007) are also available for dimensions three and lower. On the hypercube, standard tensor product nodes are constructed from one-dimensional Legendre-Gauss-Lobatto nodes. As for the basis functions, the full family of Jacobi-Gauss and Jacobi-Gauss-Lobatto quadrature points is available for specific applications.

Taken together, the functionality described thus far provides comprehensive support for interpolation, that is, the evaluation of a polynomial defined by its nodal DOFs, at any point, and further the ability to obtain any derivative of the interpolating polynomial.

DRAFT

Quadrature

While nodal degrees of freedom must be *unisolvent* or *interpolatory* (i.e. uniquely determine a function in a discrete function space), this requirement does not exist in the setting of quadrature, where the objective is to accurately approximate the value of an integral of a function on a shape. Using function values at points $\boldsymbol{x}_i$ ("nodes") and associated weights w_i, quadrature rules are generically expressed as

$$\int_D f(\boldsymbol{x})\mathrm{d}\boldsymbol{x} \approx \sum_{k=0}^{N} f(\boldsymbol{x}_k)w_k.$$

Besides the standard building blocks consisting of domains, basis functions and node sets, `modepy` also offers a wide array of quadrature rules that can be used on each domain. The quadrature rules are provided as implementations of the `Quadrature` superclass.

For the interval, a broad selection of quadrature rules are present: Clenshaw–Curtis, Fejér, and Jacobi-Gauss. The latter class includes the commonly used Chebyshev and Legendre types, among others. Lobatto (that is, endpoint-included) variants of these are also available. The line segment quadratures can be used as part of the `TensorProductQuadrature` class to define different higher-dimensional rules. Additionally, several state-of-the-art higher-dimensional rules are available:

- Grundmann–Möller (Grundmann & Möller, 1978) rules for the n-simplex.
- Vioreanu–Rokhlin (Vioreanu & Rokhlin, 2014) rules for the two- and three-dimensional simplex (see Figure 3). The quadrature nodes from these rules are also suitable for interpolation.
- Xiao–Gimbutas (Xiao & Gimbutas, 2010) rules for the two- and three-dimensional simplex.
- Jáskowiec–Sukumar (Jaśkowiec & Sukumar, 2021) rules for the tetrahedron.
- Witherden–Vincent (Witherden & Vincent, 2015) rules for the two- and three-dimensional hypercube (see Figure 3). These rules have significantly fewer nodes than an equivalent tensor product rule, while maintaining positive weights.

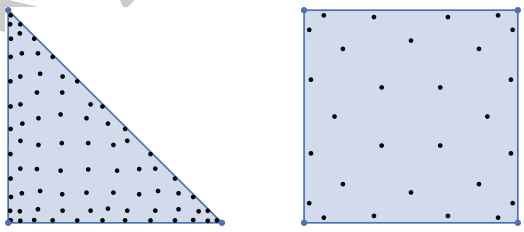

Figure 3: (left) Vioreanu–Rokhlin quadrature points of order 11 and (right) Witherden–Vincent quadrature points of order 11.

Most of the rules for simplices and hypercubes are available to high order (i.e. ≥ 20). However, `modepy` also has functionality that allows constructing desired quadrature nodes on a given domain. This can be found in `modepy.quadrature.construction`.

Matrices

While `modepy` is sufficiently flexible to accommodate different applications, it also provides functionality specific to FEM needs. In particular, it allows constructing a variety of matrix operators that define necessary resampling operators and bilinear forms used in FEM codes.

The Vandermonde matrix used in interpolation is provided by the `vandermonde` function. This can be used to obtain point-to-point interpolants using `resampling_matrix`. In a similar

DRAFT

fashion, the `differentiation_matrices` function can be used to construct derivative operators along each reference axis.

A specific bilinear form matrix can be obtained using `nodal_quadrature_bilinear_form_matrix`, which requires providing the usual test functions, trial functions and corresponding nodes for the input and output. This allows constructing very general families of bilinear forms with or without oversampling. One such example is the standard mass matrix, which can be obtained more easily using the `mass_matrix` function. If the provided functions are derivatives of the basis functions, standard weak forms of differentiation operators can also be obtained.

A Practical Example

In the following example, we demonstrate how to use the library to define a non-default tensor product shape — the prism — and construct a first derivative in the x direction on this reference element. To produce the derivative matrix operator, we go through all the objects described in the previous section: nodes, modes, and quadrature rules.

```
import numpy as np
import modepy as mp

# Define the shape on which we will operate
line = mp.Simplex(1)
triangle = mp.Simplex(2)
prism = mp.TensorProductShape((triangle, line))

assert prism.dim == 3

# Define a function space for the prism of order 12
n = 12
space = mp.TensorProductSpace((mp.PN(triangle.dim, n), mp.PN(line.dim, n)))

assert space.order == n
assert space.spatial_dim == 3
assert space.space_dim == (n + 1) * (n + 1) * (n + 2) // 2

# Define a basis function for the prism
basis = mp.orthonormal_basis_for_space(space, prism)

# Define a point set for the prism
nodes = mp.edge_clustered_nodes_for_space(space, prism)

# Define a quadrature rule for the prism
quadrature = mp.TensorProductQuadrature([
    mp.VioreanuRokhlinSimplexQuadrature(n, triangle.dim),
    mp.LegendreGaussQuadrature(n),
])

# Define a bilinear form: weak derivative in the i=x direction
i = 0
weak_d = mp.nodal_quadrature_bilinear_form_matrix(
    quadrature,
    test_functions=basis.derivatives(i),
    trial_functions=basis.functions,
    nodal_interp_functions_test=basis.functions,
    nodal_interp_functions_trial=basis.functions,
```

DRAFT

```
    input_nodes=nodes,
    output_nodes=nodes,
)
inv_mass = mp.inverse_mass_matrix(basis, nodes)

# Compute derivative and check exact solution
x = nodes[i]
f = 1.0 - np.sin(x) ** 3
f_ref = -3.0 * np.cos(x) * np.sin(x) ** 2
f_approx = inv_mass @ weak_d.T @ f

error = np.linalg.norm(f_approx - f_ref) / np.linalg.norm(f_ref)
assert error < 1.0e-4
```

Acknowledgements

A. Fikl was supported by the Office of Naval Research (ONR) as part of the Multidisciplinary University Research Initiatives (MURI) Program, under Grant Number *N00014-16-1-2617*. A. Klöckner was supported by the US National Science Foundation under award number DMS-2410943, and by the US Department of Energy under award number DE-NA0003963.

References

Bezanson, J., Edelman, A., Karpinski, S., & Shah, V. B. (2017). Julia: A fresh approach to numerical computing. *SIAM Review*, *59*, 65–98. https://doi.org/10.1137/141000671

Brenner, S., & Scott, R. (2007). *The mathematical theory of finite element methods*. Springer Science & Business Media. ISBN: 9780387759333

Čertík, O., Fernando, I., Garg, S., Rathnayake, T., & others. (2013). *SymEngine: A fast symbolic manipulation library written in c++*. https://github.com/symengine/symengine

Chan, J., Knapp, D., McCallum, M., Ranocha, H., Wang, V. X., & Markert, J. (2024). *StartUpDG.jl: Reference elements and physical meshes for DG* (Version v1.1.5). https://github.com/jlchan/StartUpDG.jl

Dubiner, M. (1991). Spectral methods on triangles and other domains. *Journal of Scientific Computing*, *6*, 345–390. https://doi.org/10.1007/bf01060030

Geuzaine, C., & Remacle, J.-F. (2009). Gmsh: A 3-d finite element mesh generator with built-in pre-and post-processing facilities. *International Journal for Numerical Methods in Engineering*, *79*(11), 1309–1331.

Glusa, C. (2021). *PyNucleus: A finite element code that specifically targets nonlocal operators*. Sandia National Lab (SNL-NM), Albuquerque, NM (United States). https://github.com/sandialabs/PyNucleus

Grundmann, A., & Möller, H. M. (1978). Invariant integration formulas for the n-simplex by combinatorial methods. *SIAM Journal on Numerical Analysis*, *15*, 282–290. https://doi.org/10.1137/0715019

Ham, D. A., Homolya, M., Kirby, R., Mitchell, L., Brubeck, P., cyruscycheng21, FAznaran, ksagiyam, Scroggs, M., Farrell, P. E., Justincrum, Ward, C., celdred, Bendall, T., Betteridge, J., & FabianL1908. (2025). *FInAT: A smarter library of finite elements* (Version Firedrake_20250331.0). Zenodo. https://doi.org/10.5281/zenodo.15114385

Homolya, M., Brubeck, P., Ham, D. A., Kirby, R., Mitchell, L., Blechta, J., Rognes, M. E., Wells, G. N., Logg, A., cyruscycheng21, Gibson, T. H., Ring, J., Yashchuk, I., Justincrum, k-b-

DRAFT

oelgaard, ksagiyam, Richardson, C., Schlömer, N., Nixon-Hill, R. W., … Scroggs, M. (2025). *Fiat: 2025.4.0* (Version 2025.4.0). Zenodo. https://doi.org/10.5281/zenodo.15302339

Isaac, T. (2020). Recursive, parameter-free, explicitly defined interpolation nodes for simplices. *SIAM Journal on Scientific Computing*, *42*, A4046–A4062. https://doi.org/10.1137/20m1321802

Jaśkowiec, J., & Sukumar, N. (2021). High-order symmetric cubature rules for tetrahedra and pyramids. *International Journal for Numerical Methods in Engineering*, *122*, 148–171. https://doi.org/10.1002/nme.6528

Klöckner, A., & others. (2025). *Grudge: An environment for discontinuous galerkin discretizations* (Version hash:193100a). https://github.com/inducer/grudge

Kloeckner, A., & others. (2025). *Pytential: Evaluate layer and volume potentials accurately* (Version hash:7395b97f). https://github.com/inducer/pytential

Kloeckner, A., Wala, M., Fernando, I., Kulkarni, K., Fikl, A., Weiner, Z., Kempf, D., Ham, D. A., Mitchell, L., Wilcox, L. C., Diener, M., Kapyshin, P., Raksi, R., & Gibson, T. H. (2024). *Pymbolic: A package to do symbolic operations for code generation* (Version v2024.2.2). Zenodo. https://doi.org/10.5281/zenodo.14526145

Meurer, A., Smith, C. P., Paprocki, M., Čertík, O., Kirpichev, S. B., Rocklin, M., Kumar, A., Ivanov, S., Moore, J. K., Singh, S., & others. (2017). SymPy: Symbolic computing in python. *PeerJ Computer Science*, *3*, e103.

Nardelli, F. Z., Belyakova, J., Pelenitsyn, A., Chung, B., Bezanson, J., & Vitek, J. (2018). Julia subtyping: A rational reconstruction. *Proceedings of the ACM on Programming Languages*, *2*, 1–27. https://doi.org/10.1145/3276483

Python Software Foundation. (2024). *ast — abstract syntax trees*. https://docs.python.org/3/library/ast.html

Schlömer, N., Papior, N., Arnold, D., Blechta, J., & Zetter, R. (2021). *QuadPy: Numerical integration (quadrature, cubature) in Python* (Version v0.16.10). Zenodo. https://doi.org/10.5281/zenodo.5541216

Trefethen, L. N. (2020). *Approximation theory and approximation practice*. Society for Industrial; Applied Mathematics. ISBN: 9781611975932

Vioreanu, B., & Rokhlin, V. (2014). Spectra of multiplication operators as a numerical tool. *SIAM Journal on Scientific Computing*, *36*, A267–a288. https://doi.org/10.1137/110860082

Warburton, T. (2007). An explicit construction of interpolation nodes on the simplex. *Journal of Engineering Mathematics*, *56*, 247–262. https://doi.org/10.1007/s10665-006-9086-6

Wicaksono, D., Acosta, U. H., Veettil, S. K. T., Kissinger, J., & Hecht, M. (2025). Minterpy: Multivariate polynomial interpolation in Python. *Journal of Open Source Software*, *10*(109), 7702. https://doi.org/10.21105/joss.07702

Witherden, F. D., & Vincent, P. E. (2015). On the identification of symmetric quadrature rules for finite element methods. *Computers & Mathematics With Applications*, *69*, 1232–1241. https://doi.org/10.1016/j.camwa.2015.03.017

Xiao, H., & Gimbutas, Z. (2010). A numerical algorithm for the construction of efficient quadrature rules in two and higher dimensions. *Computers & Mathematics With Applications*, *59*, 663–676. https://doi.org/10.1016/j.camwa.2009.10.027

DRAFT